

MODULE 4: CONCURRENCY
WEEK 4

Learning Outcomes:

After completing this module you are expected to understand the following:

1. States and state diagrams (cross-reference SF/State and State Machines)
2. Structures (ready list, process control blocks, and so forth)

A. Engage

Memes Station

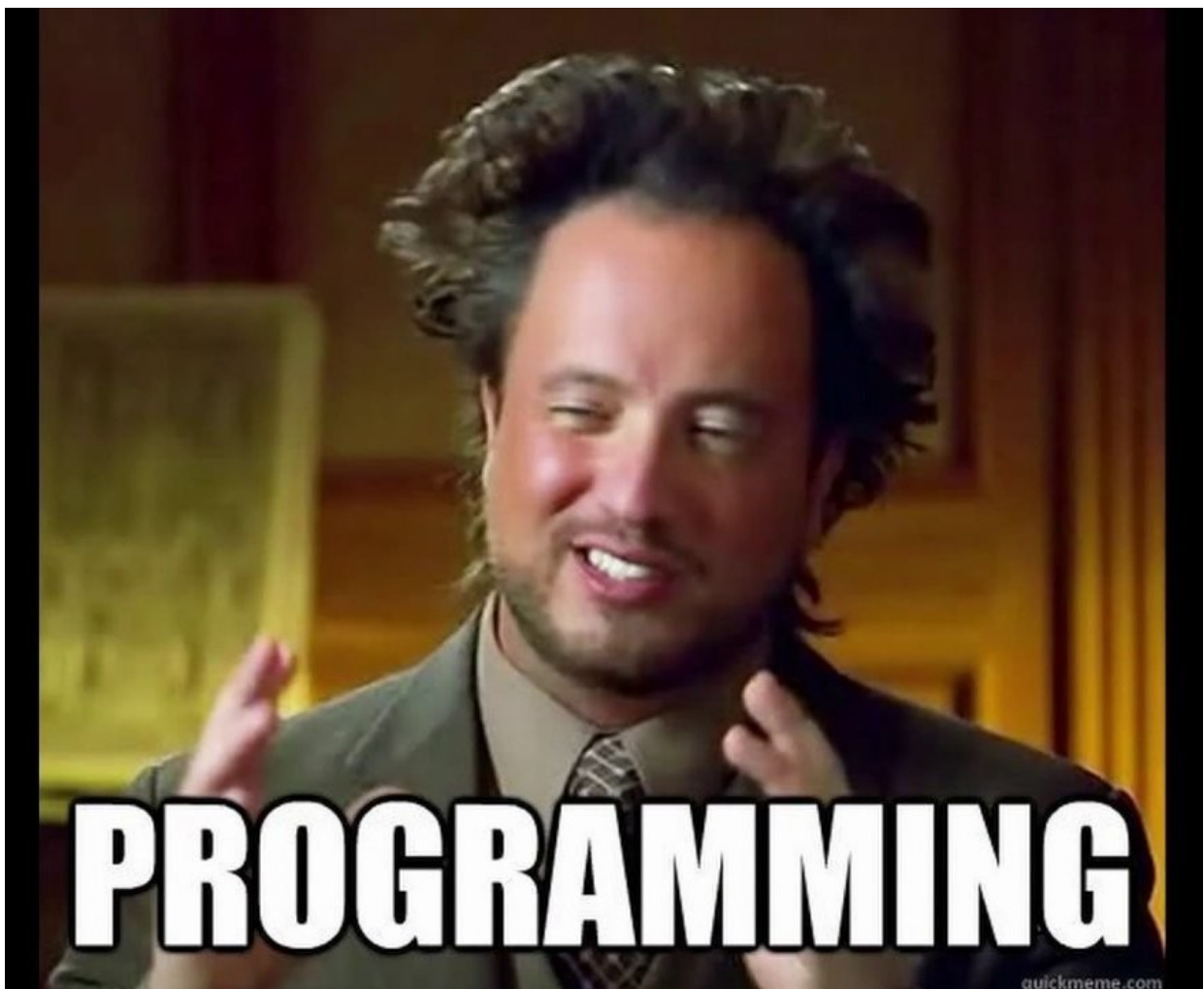


Figure 4.0: Steve Jobs (1997)

B. Explain

Computer programming is the process of designing and building an executable computer program to accomplish a specific computing result or to perform a specific task. Programming involves tasks such as: analysis, generating algorithms, profiling algorithms' accuracy and resource consumption, and the implementation of algorithms in a chosen programming language (commonly referred to as **coding**). The source code of a program is written in one or more languages that are intelligible to programmers, rather than machine code, which is directly executed by the central processing unit. The purpose of programming is to find a sequence of instructions that will automate the performance of a task (which can be as complex as an operating system) on a computer, often for solving a given problem. Proficient programming thus often requires expertise in several different subjects, including knowledge of the application domain, specialized algorithms, and formal logic.

C. Explore

Video Titles:

1. Lecture 1, unit 1: Introduction to Concurrency

YouTube Links:

1. <https://www.youtube.com/watch?v=iKtvNJQoCNw>

Module Video Filenames:

1. WEEK04_Concurrency

D. Elaborate

Introduction

1. States and state diagrams (cross-reference SF/State and State Machines)

Concurrency is the execution of the multiple instruction sequences at the same time. It happens in the operating system when there are several process threads running in parallel. The running process threads always communicate with each other through shared memory or message passing. Concurrency results in sharing of resources result in problems like deadlocks and resources starvation.

It helps in techniques like coordinating execution of processes, memory allocation and execution scheduling for maximizing throughput.

Principles of Concurrency:

Both interleaved and overlapped processes can be viewed as examples of concurrent processes, they both present the same problems.

The relative speed of execution cannot be predicted. It depends on the following:

- The activities of other processes
- The way operating system handles interrupts
- The scheduling policies of the operating system

Problems in Concurrency:

- Sharing global resources – Sharing of global resources safely is difficult. If two processes both make use of a global variable and both perform read and write on that variable, then the order in which various read and write are executed is critical.
- Optimal allocation of resources – It is difficult for the operating system to manage the allocation of resources optimally.
- Locating programming errors – It is very difficult to locate a programming error because reports are usually not reproducible.
- Locking the channel – It may be inefficient for the operating system to simply lock the channel and prevents its use by other processes.

Advantages of Concurrency:

- Running of multiple applications – It enable to run multiple applications at the same time.
- Better resource utilization – It enables that the resources that are unused by one application can be used for other applications.

- Better average response time – Without concurrency, each application has to be run to completion before the next one can be run.
- Better performance – It enables the better performance by the operating system. When one application uses only the processor and another application uses only the disk drive then the time to run both applications concurrently to completion will be shorter than the time to run each application consecutively.

Drawbacks of Concurrency:

- It is required to protect multiple applications from one another.
- It is required to coordinate multiple applications through additional mechanisms.
- Additional performance overheads and complexities in operating systems are required for switching among applications.
- Sometimes running too many applications concurrently leads to severely degraded performance.

Issues of Concurrency:

- Non-atomic – Operations that are non-atomic but interruptible by multiple processes can cause problems.
- Race conditions – A race condition occurs if the outcome depends on which of several processes gets to a point first.
- Blocking – Processes can block waiting for resources. A process could be blocked for long period of time waiting for input from a terminal. If the process is required to periodically update some data, this would be very undesirable.
- Starvation – It occurs when a process does not obtain service to progress.
- Deadlock – It occurs when two processes are blocked and hence neither can proceed to execute.

2. Structures (ready list, process control blocks, and so forth)

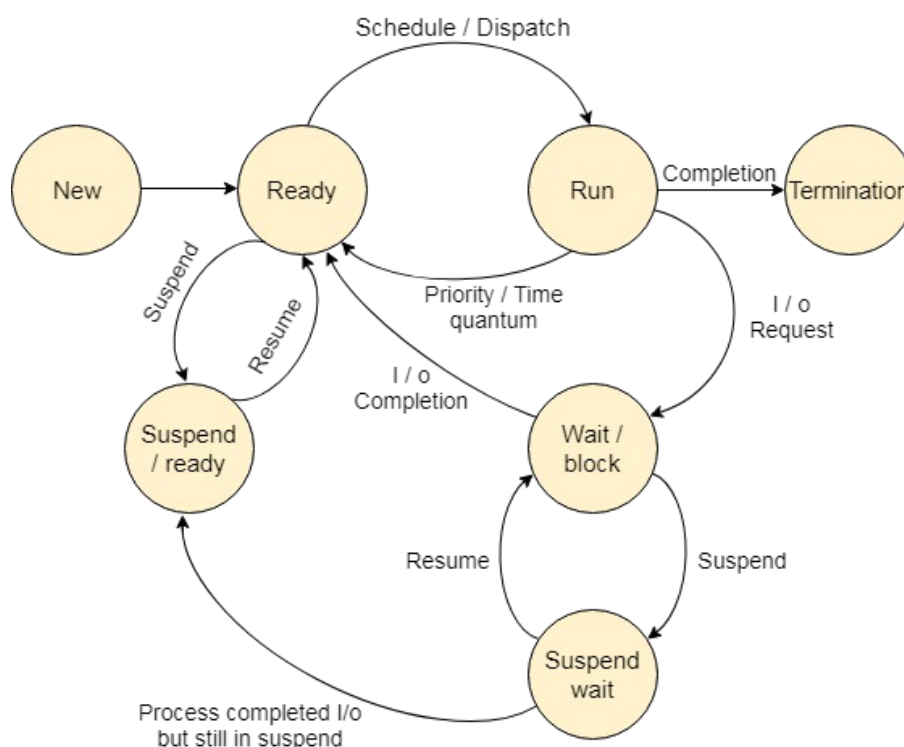


Figure 2.0. State Diagram

The process, from its creation to completion, passes through various states. The minimum number of states is five.

The names of the states are not standardized although the process may be in one of the following states during execution.

1. New - A program which is going to be picked up by the OS into the main memory is called a new process.

2. Ready - Whenever a process is created, it directly enters in the ready state, in which, it waits for the CPU to be assigned. The OS picks the new processes from the secondary memory and put all of them in the main memory.

The processes which are ready for the execution and reside in the main memory are called ready state processes. There can be many processes present in the ready state.

3. Running - One of the processes from the ready state will be chosen by the OS depending upon the scheduling algorithm. Hence, if we have only one CPU in our system, the number of running processes for a particular time will always be one. If we have n processors in the system then we can have n processes running simultaneously.

4. Block or wait - From the Running state, a process can make the transition to the block or wait state depending upon the scheduling algorithm or the intrinsic behavior of the process.

When a process waits for a certain resource to be assigned or for the input from the user then the OS move this process to the block or wait state and assigns the CPU to the other processes.

5. Completion or termination - When a process finishes its execution, it comes in the termination state. All the context of the process (Process Control Block) will also be deleted the process will be terminated by the Operating system.

6. Suspend ready - A process in the ready state, which is moved to secondary memory from the main memory due to lack of the resources (mainly primary memory) is called in the suspend ready state.

If the main memory is full and a higher priority process comes for the execution then the OS have to make the room for the process in the main memory by throwing the lower priority process out into the secondary memory. The suspend ready processes remain in the secondary memory until the main memory gets available.

7. Suspend wait - Instead of removing the process from the ready queue, it's better to remove the blocked process which is waiting for some resources in the main memory. Since it is already waiting for some resource to get available hence it is better if it waits in the secondary memory and make room for the higher priority process. These processes complete their execution once the main memory gets available and their wait is finished.

Operations on the Process

1. Creation - Once the process is created, it will be ready and come into the ready queue (main memory) and will be ready for the execution.

2. Scheduling - Out of the many processes present in the ready queue, the Operating system chooses one process and start executing it. Selecting the process which is to be executed next, is known as scheduling.

3. Execution - Once the process is scheduled for the execution, the processor starts executing it. Process may come to the blocked or wait state during the execution then in that case the processor starts executing the other processes.

4. Deletion/killing - Once the purpose of the process gets over then the OS will kill the process. The Context of the process (PCB) will be deleted and the process gets terminated by the Operating system.

E. Evaluate

ASSESSMENT:

Instructions: You may write your answer on the Answer Sheet (AS) provided on this module.

I. Identification (2-Point Each):

1. One of the processes from the ready state will be chosen by the OS depending upon the scheduling algorithm.
2. Once the purpose of the process gets over then the OS will kill the process.
3. It occurs when a process does not obtain service to progress.
4. When a process finishes its execution, it comes in the termination state.
5. A process in the ready state, which is moved to secondary memory from the main memory due to lack of the resources (mainly primary memory) is called in the suspend ready state.
6. Instead of removing the process from the ready queue, it's better to remove the blocked process which is waiting for some resources in the main memory.
7. Once the process is created, it will be ready and come into the ready queue (main memory) and will be ready for the execution.
8. Out of the many processes present in the ready queue, the Operating system chooses one process and start executing it.
9. From the Running state, a process can make the transition to the block or wait state depending upon the scheduling algorithm or the intrinsic behavior of the process.
10. Once the process is scheduled for the execution, the processor starts executing it.

Laboratory Activity

Task 4.1

Instruction: Copy the source code and paste it to your prepared ide and save it with a file named "Shortest.cpp".

AIM: To write a program to stimulate the CPU scheduling algorithm Shortest job first (Non- Preemption)

DESCRIPTION:

To calculate the average waiting time in the shortest job first algorithm the sorting of the process based on their burst time in ascending order then calculate the waiting time of each process as the sum of the bursting times of all the process previous or before to that process.

ALGORITHM:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue

Step 3: For each process in the ready Q, assign the process id and accept the CPU burst time

Step 4: Start the Ready Q according the shortest Burst time by sorting according to lowest to highest burst time.

Step 5: Set the waiting time of the first process as 0 and its turnaround time as its burst time.

Step 6: Sort the processes names based on their Burst time Step 7: For each process in the ready queue, calculate

- a) $\text{Waiting time}(n) = \text{waiting time}(n-1) + \text{Burst time}(n-1)$
- b) $\text{Turnaround time}(n) = \text{waiting time}(n) + \text{Burst time}(n)$

Step 8: Calculate

- c) $\text{Average waiting time} = \text{Total waiting Time} / \text{Number of process}$
 - d) $\text{Average Turnaround time} = \text{Total Turnaround Time} / \text{Number of process}$
- Step 9: Stop the process

Output:

Enter the number of processes -- 4
Enter Burst Time for Process 0 -- 8
Enter Burst Time for Process 1 -- 4
Enter Burst Time for Process 2 -- 2
Enter Burst Time for Process 3 -- 2

PROCESS	BURST TIME	WAITING TIME	TURNAROUND TIME
P2	2	0	2
P3	2	2	4
P1	4	4	8
P0	8	8	16

Average Waiting Time -- 3.500000

Average Turnaround Time -- 7.500000

References:

- 1. **Concurrency in Operating System–**
<https://bit.ly/3pRzVF4>
- 2. **Process States–**
<https://www.javatpoint.com/os-process-states>

Facilitated By:

Name	:	
MS Teams Account (email)	:	
Facebook Page	:	